

Intents & Activity Navigation

Screens, the back stack, and how data travels between them

by Ekramul Haque Amin

Lecturer, DIU

1

Intents

2

Activity Stack

3

Passing Data

4

putExtra Types

What Is an Intent?

Simple idea, real-life example

An Intent is a **message** that tells Android:

- “Do this action”, or
- “Open this screen for me”

It carries the request from one part of the app to another — like handing a note to someone.



Think of it like ordering food

You (Activity A) tell the waiter (Intent): “Bring me a pizza (open Activity B)”.

You don’t need to know how the kitchen works — you just send the request, and the right thing happens.

Key idea: Intents let Activities, Services, and Broadcast Receivers communicate — without knowing each other’s code.

Two Types of Intents

Explicit Intent vs. Implicit Intent

E Explicit Intent

You name the exact screen (class) to open.
Used for navigation inside your OWN app.

```
Intent intent = new Intent(MainActivity.this,  
    SecondActivity.class);  
  
startActivity(intent);
```

I Implicit Intent

You describe an ACTION you want done.
Android finds an app that can do it.

```
Intent emailIntent = new Intent(Intent.ACTION_SEND);  
emailIntent.setType("message/rfc822");  
  
Intent chooserIntent = Intent.createChooser(emailIntent,  
    "Send Email");  
  
startActivity(chooserIntent);
```

Navigating Between Activities

The 3-step pattern you will use again and again



Going back: The Back button (or `finish()` in code) closes the current Activity and returns to the previous one — more on this next.

The Activity Stack (Back Stack)

Android remembers your screens in a stack — Last In, First Out (LIFO)

- A new Activity is placed on top of a stack.
- Like a stack of plates — add or remove only from the top. That's **LIFO**.
- Only the **TOP** Activity is visible and active.
- Activities below are paused in the background.

↓ *visible to the user*

EditNoteActivity (TOP — visible now)

DetailActivity (paused)

MainActivity (bottom — opened first)

Rule: New screens are pushed to the top. Pressing Back pops the top screen off, revealing the one below.

Stack in Action: Push & Pop

startActivity() pushes a screen on; Back / finish() pops it off



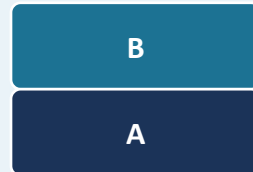
PUSH — startActivity()

Opening a new screen adds it to the top of the stack.

Before



After



*B is now on top
(visible)*

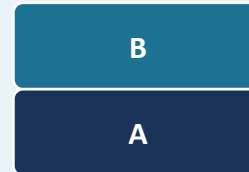
```
Intent intent = new Intent(this,  
  
    ActivityB.class);  
  
startActivity(intent);
```



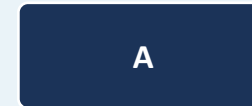
POP — Back / finish()

Going back removes the top screen; the one below is shown again.

Before



After



*B is removed
(destroyed)*

```
// user taps system Back button  
  
// or in code:  
  
finish();
```

Ways to Share Data Between Screens

Pick the right tool depending on how long the data should live

Intent Extras

```
putExtra() / getExtra()
```

Send data directly when opening a new screen. Gone once both screens close.

Bundle

```
putExtras() / getExtras()
```

Groups many key-value pairs into one package — travels inside an Intent.

SharedPreferences

```
key-value, persistent
```

Small settings/data that stay saved even after the app is closed and reopened.

Database

```
SQLite / Room
```

Structured, larger data (e.g. a full notes list) that must be stored and queried.

Sending Data: putExtra() and Bundle

Send extras one at a time, or grouped together in a Bundle

1 Directly on the Intent

```
Intent intent = new Intent(this, SecondActivity.class);
intent.putExtra("note_title", titleText);
intent.putExtra("note_body", bodyText);
startActivity(intent);
```

2 Grouped in a Bundle

```
Bundle bundle = new Bundle();
bundle.putString("note_title", titleText);
bundle.putInt("note_color", 2);
intent.putExtras(bundle);
startActivity(intent);
// receiving side:
Bundle b = getIntent().getExtras();
```

putExtra() Is Overloaded

Same method name, many versions — Java picks the right one by data type

Function overloading = same method name, different parameter types. Android provides a matching `putExtra()` for almost every data type. Each call stores a **key–value pair** inside the Intent — the key is a String label, the value is the data.

Data Type	Example putExtra() Call
String	<code>intent.putExtra("name", "Ali");</code>
int	<code>intent.putExtra("age", 21);</code>
boolean	<code>intent.putExtra("isLoggedIn", true);</code>
float / double	<code>intent.putExtra("price", 19.99);</code>
long	<code>intent.putExtra("timestamp", 123456789L);</code>
String[] / int[]	<code>intent.putExtra("tags", new String[]{"a","b"});</code>
Serializable / Parcelable	<code>intent.putExtra("user", userObject);</code>

Reading Extras Safely: `getStringExtra()`

Matching get methods — but primitive types need a default value

?

Object types — no default needed

String, Bundle, ArrayList... are objects, so they can safely be null if the key is missing.

```
String name = intent.getStringExtra("username");  
// name is null if not found
```

0

Primitive types — default required

int, float, boolean, long, double CANNOT be null — you must supply a fallback value.

```
int age = intent.getIntExtra("age", 0);  
boolean ok = intent.getBooleanExtra("isLoggedIn", false);
```

Rule of thumb: Objects → can return null. Primitives (int, float, boolean, long, double) → always pass a default value as the 2nd argument.

SharedPreferences: Data That Survives Restarts

Same key-value idea as Intent extras — but saved permanently on the device

S

Saving a value

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = prefs.edit();
editor.putString("username", "Ayesha");
editor.apply();
```

G

Getting a value back

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);
String name = prefs.getString("username", "Guest");
```

"Guest" is the default — same rule as `getIntExtra()`: always give a fallback for a missing key.

Bigger, structured data (like a full notes list or user table) instead goes into a Database — SQLite or Room

Recap:

What we learned today

Intent

A message that requests an action or opens another screen

Explicit / Implicit

Name the exact screen vs. describe an action for Android to match

Activity Stack

Screens are pushed on top; Back / finish() pops the top one off (LIFO)

putExtra() Overload

One method name, many versions — one per data type, as key–value pairs

getStringExtra() Defaults

Primitives (int, float, boolean) need a default; objects can return null

Bundle / SharedPreferences

Group extras together, or save small data that survives app restarts